UNIT - V

GRAPHS AND TREES

CO5: Demonstrate basic operations on trees and graphs.

5.1 GRAPH INTRODUCTION [April 22, 24, Oct. 24]

A graph is a set of pair (V, E), where V is the set of vertices and E is the set of edges, connecting the pairs of vertices.

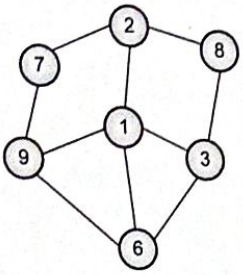


Fig. 5.1

In the above graph,

V = {1, 2, 3, 6, 7, 8, 9}

E = {12, 13, 19, 16, 27, 28, 79, 83, 96, 36}

5.1.1 Graph Terms and Properties

- **Adjacency:** Two node or vertices are adjacent if they are connected to each other through an edge. In the above example, 1 is adjacent to 2, 3 is adjacent to 1, and so on.
- **Path:** Path represents a sequence of edges between the two vertices. In the above example, 1382 represents a path from 1 to 2.
- **Self-Loop:** Is an edge that connects a vertex to itself. A simple graph contains no loops.
- **Multi Edge:** Two or more edges that are connecting to the same two vertices.
- **Simple Graph:** Graphs without loops or parallel edges are called simple graphs.
- **The Degree of a Node:** The degree of a node is the number of edges incident/attached on it.
- **Path:** A path can be defined as the sequence of nodes that are followed in order to reach some terminal node E from the initial node A.
- **Simple Path:** A path is a Simple path if no vertices(and thus edges) are not repeated.
- **Cycle:** A cycle can be defined as the path which has no repeated edges or vertices except the first and last vertices.

Some Graph Types: Weighted vs. Unweighted Graphs:

A weight is a numerical value attached to each individual edge in the graph. Weighted Graph will contain weight on each edge whereas unweighted does not.

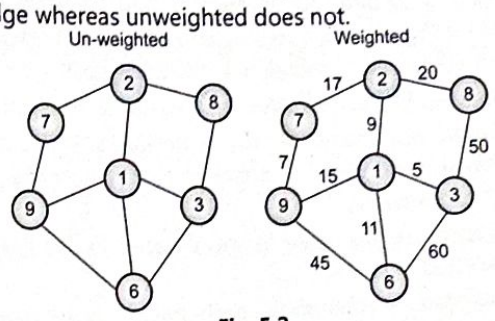


Fig. 5.2

5.1.2 Directed (Digraph) vs. Undirected Graph (Storage Representation of Graph)

Directed (Digraph): A directed graph is a set of vertices (nodes) connected by edges, with each node having a direction associated with it. Edges are usually represented by arrows pointing in the direction the graph can be traversed.

Undirected Graphs: In an undirected graph the edges are bidirectional, with no direction associated with them. Hence, the graph can be traversed in either direction. The absence of an arrow tells us that the graph is undirected.

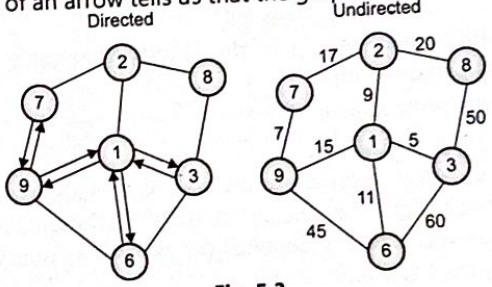


Fig. 5.3

Graph Applications:

- Dijkstra's Algorithm
- Prims's Algorithm
- Kruskal's Algorithm
- Graphs are used to define the flow of computation.
- Graphs are used to represent networks of communication.
- Graphs are used to represent data organization.
- Graph theory is used to find shortest path in road or a network.

In Google Maps, various locations are represented as vertices or nodes and the roads are represented as edges and graph theory is used to find the shortest path between two nodes.

5.2 GRAPH USING ADJENCY MATRIX AND LIST

A graph is a data structure that consists of the following two components:

(i) A finite set of vertices also called as nodes.

(ii) A finite set of ordered pair of the form (u, v) called as edge.

The pair is ordered because (u, v) is not the same as (v, u) in case of a directed graph (digraph). The pair of the form (u, v) indicates that there is an edge from vertex u to vertex v. The edges may contain weight/value/cost.

Graphs are used to represent many real-life applications:

- Graphs are used to represent networks. The networks may include paths in a city or telephone network or circuit network.
- Graphs are also used in social networks like linkedIn, Facebook.

For example, in Facebook, each person is represented with a vertex(or node). Each node is a structure and contains information like person id, name, gender, and locale. See this for more applications of graph.

Following is an example of an undirected graph with five vertices.

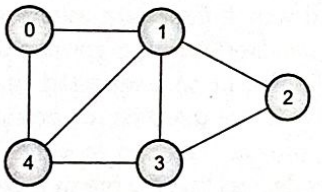


Fig. 5.4

The following two are the most commonly used representations of a graph.

(i) Adjacency Matrix

(ii) Adjacency List

There are other representations also like, Incidence Matrix and Incidence List. The choice of graph representation is situation specific. It totally depends on the type of operations to be performed and ease of use.

5.2.1 Adjacency Matrix

Adjacency Matrix is a 2D array of size V × V where V is the number of vertices in a graph. Let the 2D array be adj[][], a slot adj[i][j] = 1 indicates that there is an edge from vertex i to vertex j. Adjacency matrix for undirected graph is always symmetric. Adjacency Matrix is also used to represent weighted graphs. If adj[i][j] = w, then there is an edge from vertex i to vertex j with weight w.

The adjacency matrix for the above example graph is:

	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	1	1
2	0	1	0	1	0
3	0	1	1	0	1
4	1	1	0	1	0

Advantage

- Representation is easier to implement and fo... Removing an edge takes O(1) time. Queries ... whether there is an edge from vertex 'u' to verte... are efficient and can be done O(1).

Disadvantage

- Consumes more space $O(V^2)$. Even if the grap... sparse(contains less number of edges), it consu... the same space.
- Adding a vertex is $O(V^2)$ time. Computing ... neighbors of a vertex takes O(V) time (Not efficien...

Please see this for a sample Python implementation ... adjacency matrix.

Program 5.1:

```
def main():
    n, m = map(int, input().split())
    adjMat = [[0] * (n + 1) for _ in range(n + 1)]

    for _ in range(m):
        u, v = map(int, input().split())
        adjMat[u][v] = 1
        adjMat[v][u] = 1

    # Optionally print adjacency matrix
    # for row in adjMat[1:]:
    #     print(' '.join(map(str, row[1:])))

if __name__ == "__main__":
    main()
```

5.2.2 Adjacency List

An array of lists is used. The size of the array is equal t... the number of vertices. Let the array be an array[]. An ent... array[i] represents the list of vertices adjacent to the ith verte... This representation can also be used to represent a weighte... graph. The weights of edges can be represented as lists o... pairs. Following is the adjacency list representation of th... below graph.

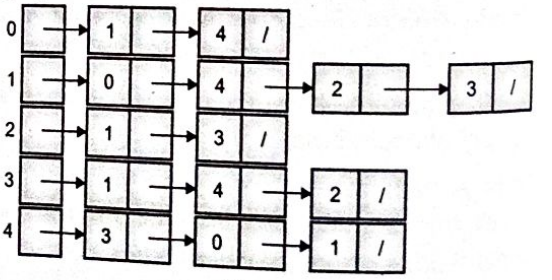


Fig. 5.5

Program 5.2: Recommended Practice.

```
def add_edge(adj, u, v):
    adj[u].append(v)
    adj[v].append(u)
def print_graph(adj):
```

```
for v in range(len(adj)):
    print(f"\n Adjacency list of vertex {v}")
    print(" head", end="")
    for x in adj[v]:
        print(f" -> {x}", end="")
    print()

def main():
    V = 5
    adj = [[] for _ in range(V)]
    add_edge(adj, 0, 1)
    add_edge(adj, 0, 4)
    add_edge(adj, 1, 2)
    add_edge(adj, 1, 3)
    add_edge(adj, 1, 4)
    add_edge(adj, 2, 3)
    add_edge(adj, 3, 4)
    print_graph(adj)
if __name__ == "__main__":
    main()
```

Advantage

- Saves space O(|V|+|E|). In the worst case, there can be C(V, 2) number of edges in a graph, thus consuming $O(V^2)$ space. Adding a vertex is easier. Computing all neighbors of a vertex takes optimal time.

Disadvantage

- Queries like whether there is an edge from vertex u to vertex V are not efficient and can be done O(V).

In Real-life problems, graphs are sparse ($|E| << |V|^2$). That is why adjacency lists Data structure is commonly used for storing graphs. Adjacency matrix will enforce ($|V|^2$) bound on time complexity for such algorithms.

5.3 BREADTH FIRST SEARCH

[April 22, 23, 24, Oct. 24]

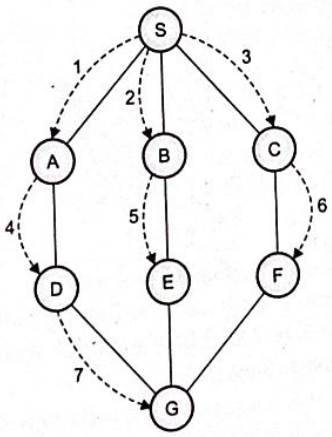


Fig. 5.6

Breadth First Search (BFS) algorithm traverses a graph in a breadthward motion and uses a queue to remember to get the next vertex to start a search, when a dead end occurs in any iteration.

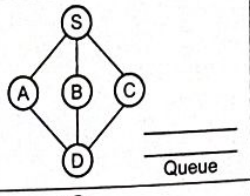
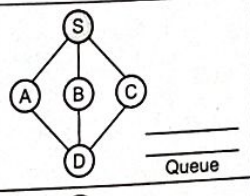
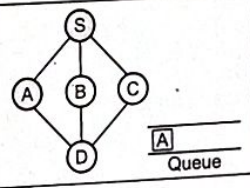
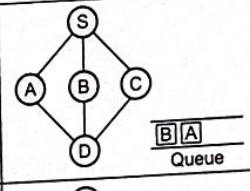
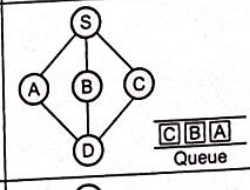
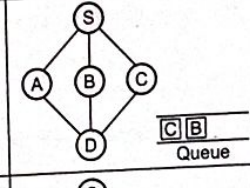
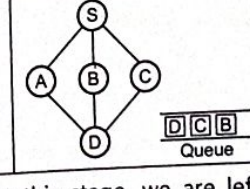
As in the example given above, BFS algorithm traverses from A to B to E to F first then to C and G lastly to D. It employs the following rules.

Rule 1: Visit the adjacent unvisited vertex. Mark it as visited. Display it. Insert it in a queue.

Rule 2: If no adjacent vertex is found, remove the first vertex from the queue.

Rule 3: Repeat Rule 1 and Rule 2 until the queue is empty.

Table 5.1

Step	Traversal	Description
1.		Initialize the queue.
2.		We start from visiting S (starting node), and mark it as visited.
3.		We then see an unvisited adjacent node from S. In this example, we have three nodes but alphabetically we choose A, mark it as visited and enqueue it.
4.		Next, the unvisited adjacent node from S is B. We mark it as visited and enqueue it.
5.		Next, the unvisited adjacent node from S is C. We mark it as visited and enqueue it.
6.		Now, S is left with no unvisited adjacent nodes. So, we dequeue and find A.
7.		From A we have D as unvisited adjacent node. We mark it as visited and enqueue it.

At this stage, we are left with no unmarked (unvisited) nodes. But as per the algorithm we keep on dequeuing in order to get all unvisited nodes. When the queue gets emptied, the program is over.

Program 5.3: Implementation.

```
MAX = 5

class Vertex:
    def __init__(self, label):
        self.label = label
        self.visited = False
# Queue implementation
queue = []
vertex_list = []
adj_matrix = [[0]*MAX for _ in range(MAX)]
vertex_count = 0

def add_vertex(label):
    global vertex_count
    vertex_list.append(Vertex(label))
    vertex_count += 1

def add_edge(start, end):
    adj_matrix[start][end] = 1
    adj_matrix[end][start] = 1

def display_vertex(vertex_index):
    print(vertex_list[vertex_index].label, end=" ")

def get_adj_unvisited_vertex(vertex_index):
    for i in range(vertex_count):
        if adj_matrix[vertex_index][i] == 1 and not
vertex_list[i].visited:
            return i
    return -1

def breadth_first_search():
    vertex_list[0].visited = True
    display_vertex(0)
    queue.append(0)

    while queue:
        current_vertex = queue.pop(0)
        adj_vertex = get_adj_unvisited_vertex(current_vertex)
        while adj_vertex != -1:
            vertex_list[adj_vertex].visited = True
            display_vertex(adj_vertex)
            queue.append(adj_vertex)
            adj_vertex = get_adj_unvisited_vertex(current_vertex)

    # Reset visited flags
    for vertex in vertex_list:
        vertex.visited = False

def main():
    for i in range(MAX):
        for j in range(MAX):
            adj_matrix[i][j] = 0
    add_vertex('S') # 0
    add_vertex('A') # 1
    add_vertex('B') # 2
    add_vertex('C') # 3
    add_vertex('D') # 4

    add_edge(0, 1) # S - A
    add_edge(0, 2) # S - B
    add_edge(0, 3) # S - C
    add_edge(1, 4) # A - D
    add_edge(2, 4) # B - D
    add_edge(3, 4) # C - D
    print("\nBreadth First Search: ", end="")
    breadth_first_search()
    print()

if __name__ == "__main__":
    main()
```

5.4 DEPTH FIRST SEARCH

[April 22, 23, 24, Oct. 2

Depth First Search (DFS) algorithm traverses a graph in a depthward motion and uses a stack to remember to get the next vertex to start a search, when a dead end occurs in any iteration.

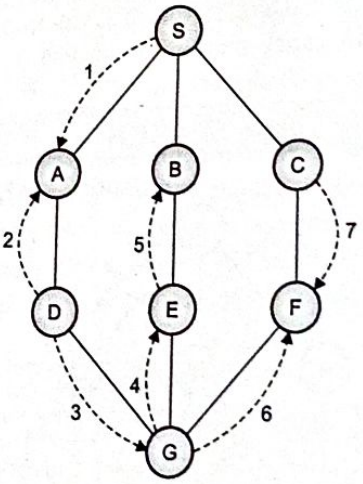


Fig. 5.7

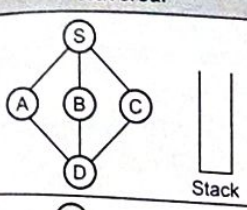
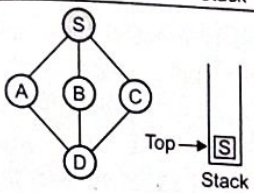
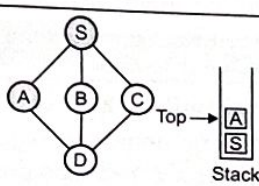
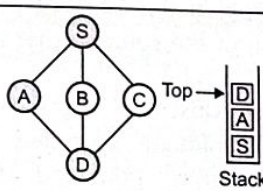
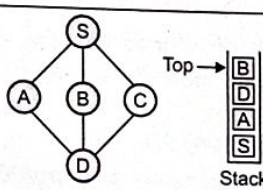
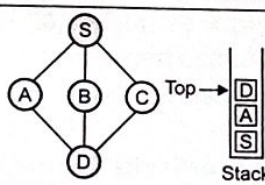
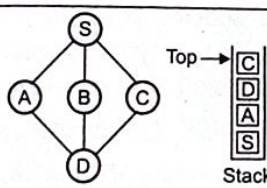
As in the example given above, DFS algorithm traverses from S to A to D to G to E to B first, then to F and lastly to C. It employs the following rules.

Rule 1: Visit the adjacent unvisited vertex. Mark it as visited. Display it. Push it in a stack.

Rule 2: If no adjacent vertex is found, pop up a vertex from the stack. (It will pop up all the vertices from the stack which do not have adjacent vertices.)

Rule 3: Repeat Rule 1 and Rule 2 until the stack is empty.

Table 5.2

Step	Traversal	Description
1.		Initialize the stack.
2.		Mark S as visited and put it onto the stack. Explore any unvisited adjacent node from S. We have three nodes and we can pick any of them. For this example, we shall take the node in an alphabetical order.
3.		Mark A as visited and put it onto the stack. Explore any unvisited adjacent node from A. Both S and D are adjacent to A but we are concerned for unvisited nodes only.
4.		Visit D and mark it as visited and put onto the stack. Here, we have B and C nodes, which are adjacent to D and both are unvisited. However, we shall again choose in an alphabetical order.
5.		We choose B, mark it as visited and put onto the stack. Here B does not have any unvisited adjacent node. So, we pop B from the stack.
6.		We check the stack top for return to the previous node and check if it has any unvisited nodes. Here, we find D to be on the top of the stack.
7.		Only unvisited adjacent node is from D is C now. So we visit C, mark it as visited and put it onto the stack.

As C does not have any unvisited adjacent node so we keep popping the stack until we find a node that has an unvisited adjacent node. In this case, there is none and we keep popping until the stack is empty.

Program 5.4: Implementation.

```
MAX = 5

class Vertex:
  def __init__(self, label):
    self.label = label
    self.visited = False

stack = []
lstVertices = []
adjMatrix = [[0]*MAX for _ in range(MAX)]
vertexCount = 0

def push(item):
  stack.append(item)

def pop():
  return stack.pop()

def peek():
  return stack[-1]

def isStackEmpty():
  return len(stack) == 0

def addVertex(label):
  global vertexCount
  vertex = Vertex(label)
  lstVertices.append(vertex)
  vertexCount += 1

def addEdge(start, end):
  adjMatrix[start][end] = 1
  adjMatrix[end][start] = 1

def displayVertex(vertexIndex):
  print(lstVertices[vertexIndex].label, end=" ")

def getAdjUnvisitedVertex(vertexIndex):
  for i in range(vertexCount):
    if adjMatrix[vertexIndex][i] == 1 and not
lstVertices[i].visited:
      return i
  return -1

def depthFirstSearch():
  lstVertices[0].visited = True
  displayVertex(0)
  push(0)

  while not isStackEmpty():
    unvisitedVertex = getAdjUnvisitedVertex(peek())
    if unvisitedVertex == -1:
      pop()
```

```
        else:
            lstVertices[unvisitedVertex].visited = True
            displayVertex(unvisitedVertex)
            push(unvisitedVertex)

    for vertex in lstVertices:
        vertex.visited = False

def main():
    global adjMatrix
    for i in range(MAX):
        for j in range(MAX):
            adjMatrix[i][j] = 0

    addVertex('S') # 0
    addVertex('A') # 1
    addVertex('B') # 2
    addVertex('C') # 3
    addVertex('D') # 4

    addEdge(0, 1) # S - A
    addEdge(0, 2) # S - B
    addEdge(0, 3) # S - C
    addEdge(1, 4) # A - D
    addEdge(2, 4) # B - D
    addEdge(3, 4) # C - D
    print("Depth First Search: ", end="")
    depthFirstSearch()
    print()

if __name__ == "__main__":
    main()
```

5.5 MINIMUM SPANNING TREE

[April 22, 24, Oct. 23, 24]

The Minimum Spanning Tree (MST) is the collection of edges required to connect all vertices in an undirected graph, with the minimum total edge weight. The animation above runs Prim's algorithm to find the MST. Another way to find the MST, which also works for unconnected graphs, is to run Kruskal's algorithm.

It is called a Minimum Spanning Tree, because it is a connected, acyclic, undirected graph, which is the definition of a tree data structure. In the real world, finding the Minimum Spanning Tree can help us find the most effective way to connect houses to the internet or to the electrical grid, or it can help us finding the fastest route to deliver packages.

An MST Thought Experiment

Let's imagine that the circles in the animation above are villages that are without electrical power, and you want to connect them to the electrical grid. After one village is given electrical power, the electrical cables must be spread out from that village to the others. The villages can be connected in a lot of different ways, each route having a different cost.

The electrical cables are expensive, and digging ditches for the cables, or stretching the cables in the air is expensive as well. The terrain can certainly be a challenge, and then there is perhaps a future cost for maintenance that is different depending on where the cables end up.

All these route costs can be factored in as edge weights in a graph. Every vertex represents a village, and every edge represents a possible route for the electrical cable between two villages. After such a graph is created, the Minimum Spanning Tree (MST) can be found, and that will be the most effective way to connect these villages to the electrical grid.

And this is actually what the first MST algorithm (Borůvka's algorithm) was made for in 1926: To find the best way to connect the historical region of Moravia, in the Check Republic, to the electrical grid.

5.6 GREEDY ALGORITHMS

[April 22, 24, Oct. 23, 24]

In computer science, a greedy algorithm is an algorithm that finds a solution to problems in the shortest time possible. It picks the path that seems optimal at the moment without regard for the overall optimization of the solution that would be formed.

Edsger Dijkstra, a computer scientist and mathematician who wanted to calculate a minimum spanning tree, introduced the term "Greedy algorithm". Prim and Kruskal came up with optimization techniques for minimizing cost of graphs.

Many Greedy algorithms were developed to solve graph problems. A graph is a structure made up of edges and vertices.

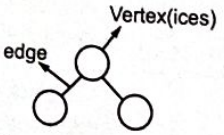


Fig. 5.8: Simple graph

Greedy vs Not Greedy Algorithms

An algorithm is greedy when the path picked is regarded as the best option based on a specific criterion without considering future consequences. But it typically evaluates feasibility before making a final decision. The correctness of the solution depends on the problem and criteria used.

Example:

A graph has various weights and you are to determine the maximum value in the tree. You'd start by searching each node and checking its weight to see if it is the largest value.

There are two approaches to solving this problem: greedy approach or not greedy.

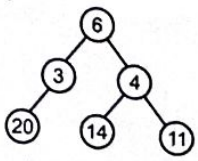


Fig. 5.9: Example graph

This graph consists of different weights and we need to find the maximum value. We'll apply the two approaches on the graph to get the solution.

Greedy Approach

In the images below, a graph has different numbers in its vertices and the algorithm is meant to select the vertex with the largest number.

Starting from vertex 6, then it's faced with two decisions – which is bigger, 3 or 4? The algorithm picks 4, and then is faced with another decision – which is bigger, 14 or 11. It selects 14, and the algorithm ends.

On the other hand there is a vertex labeled 20 but it is attached to vertex 3 which greedy does not consider as the best choice. It is important to select appropriate criteria for making each immediate decision.

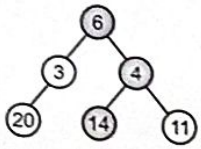


Fig. 5.10: The sample graph showing the greedy approach

Not Greedy Approach

The "not greedy" approach checks all options before arriving at a final solution, unlike the "greedy approach" which stops once it gets its results.

Starting from vertex 6, then it's faced with two decisions – which is bigger, 3 or 4? The algorithm picks 4, and then is faced with another decision – which is bigger, 14 or 11. It selects 14 and keeps it aside.

Then it runs the process again, starting from vertex 6. It selects the vertex with 3 and checks it. 20 is attached to the vertex 3 and the process stops. Now it compares the two results – 20 and 14. 20 is bigger, so it selects the vertex (3) that carries the largest number and the process ends.

This approach considers many possibilities in finding the better solution.

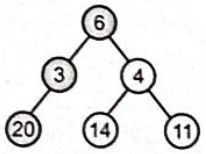


Fig. 5.11: The sample graph showing the non-greedy approach

Characteristics of a Greedy Algorithm

- The algorithm solves its problem by finding an optimal solution. This solution can be a maximum or minimum value. It makes choices based on the best option available.
- The algorithm is fast and efficient with time complexity of O(n log n) or O(n). Therefore, applied in solving large-scale problems.
- The search for optimal solution is done without repetition, the algorithm runs once.
- It is straightforward and easy to implement.

How to Use Greedy Algorithms?

Before applying a greedy algorithm to a problem, you need to ask two questions:

1. Do you need the best option at the moment from the problem?

2. Do you need an optimal solution (either minimum or maximum value)?

If your answer to these questions is "Yes", then a greedy algorithm is a good choice to solve your problem.

Procedure

Let's assume you have a problem with a set of numbers and you need to find the minimum value.

You start of by defining the constraint, which in this case is finding the minimum value. Then each number will be scanned and checked on each constraint which serves as a condition to be fulfilled. If the condition is true, the number(s) is selected and returned as the final solution.

Here's a flowchart representation of this process:

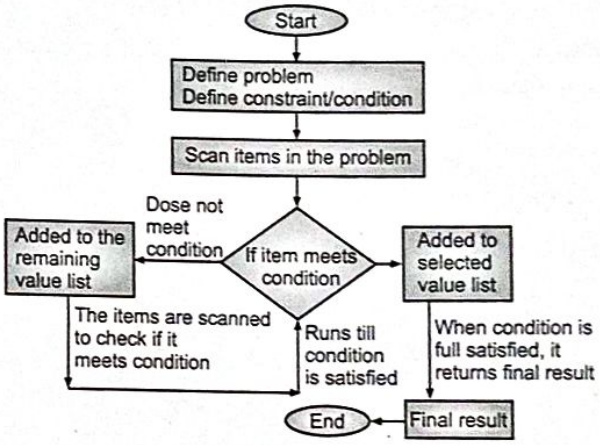


Fig. 5.12: Flow chart showing the process for solving a problem using greedy algorithms

5.7 PRIMS ALGORITHM [April 22, 24, Oct. 23, 24]

How Prim's Algorithm Works?

It falls under a class of algorithms called greedy algorithms that find the local optimum in the hopes of finding a global optimum. We start from one vertex and keep adding edges with the lowest weight, until we reach to our goal.

The steps for implementing Prim's algorithm are as follows:

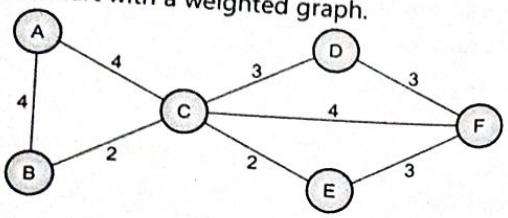
Step 1: Initialize the minimum spanning tree with a vertex chosen at random.

Step 2: Find all the edges that connect the tree to new vertices, find the minimum and add it to the tree.

Step 3: Keep repeating step 2 until we get a minimum spanning tree.

5.7.1 Example of Prim's Algorithm

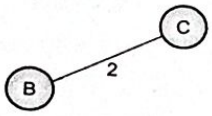
Step 1: Start with a weighted graph.



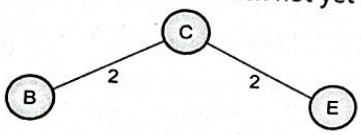
Step 2: Choose a vertex.



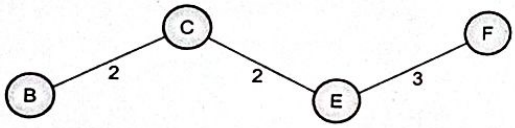
Step 3: Choose the shortest edge from this vertex and add it.



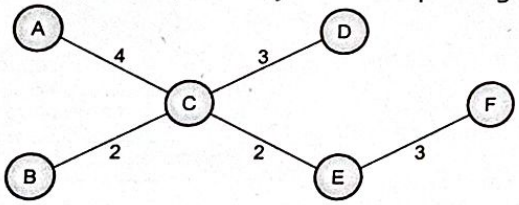
Step 4: Choose the nearest vertex not yet in the solution.



Step 5: Choose the nearest edge not yet in the solution, if there are multiple choices, choose one at random.



Step 6: Repeat step 3 until you have a spanning tree.



Prim's Algorithm Pseudocode

The pseudocode for prim's algorithm shows how we create two sets of vertices U and V-U? U contains the list of vertices that have been visited and V-U the list of vertices that have not. One by one, we move vertices from set V-U to set U by connecting the least weight edge.

Algorithm 5.1: Prim's Algorithm.

```
T = ϕ;
U = { 1 };
while (U ≠ V)
 let (u, v) be the lowest cost edge such that u ∈ U and v ∈ V - U;
  T = T ∪ {(u, v)}
  U = U ∪ {v}
```

Although adjacency matrix representation of graphs is used, this algorithm can also be implemented using Adjacency List to improve its efficiency.

Program 5.5: Implementation.

```
INF = 9999999
V = 5

G = [
  [0, 9, 75, 0, 0],
  [9, 0, 95, 19, 42],
  [75, 95, 0, 51, 66],
  [0, 19, 51, 0, 31],
  [0, 42, 66, 31, 0]
]
def prim():
  selected = [False] * V
  no_edge = 0
  selected[0] = True

  print("Edge : Weight")

  while no_edge < V - 1:
    minimum = INF
    x = 0
    y = 0
    for i in range(V):
      if selected[i]:
        for j in range(V):
          if not selected[j] and G[i][j] != 0:
            if minimum > G[i][j]:
              minimum = G[i][j]
              x = i
              y = j
    print(f"{x} - {y} : {G[x][y]}")
    selected[y] = True
    no_edge += 1

if __name__ == "__main__":
  prim()
```

5.8 KRUSKAL'S ALGORITHM

[April 22, 24, Oct. 23, 24]

It falls under a class of algorithms called greedy algorithms that find the local optimum in the hopes of finding a global optimum. We start from the edges with the lowest weight and keep adding edges until we reach our goal.

The steps for implementing Kruskal's algorithm are as follows:

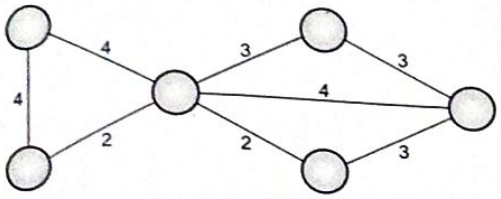
Step 1: Sort all the edges from low weight to high

Step 2: Take the edge with the lowest weight and add it to the spanning tree. If adding the edge created a cycle, then reject this edge.

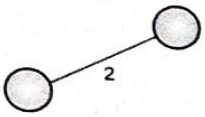
Step 3: Keep adding edges until we reach all vertices.

5.8.1 Example of Kruskal's Algorithm

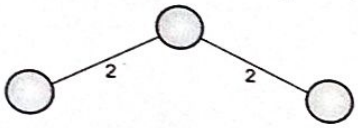
Step 1: Start with a weighted graph.



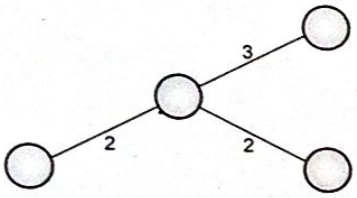
Step 2: Choose the edge with the least weight, if there are more than 1, choose anyone.



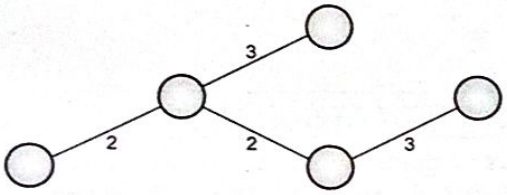
Step 3: Choose the next shortest edge and add it



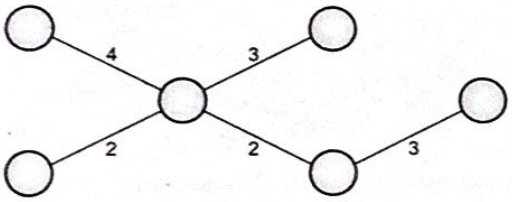
Step 4: Choose the next shortest edge that doesn't create a cycle and add it.



Step 5: Choose the next shortest edge that doesn't create a cycle and add it.



Step 6: Repeat steps until you have a spanning tree.



Kruskal Algorithm Pseudocode

Any minimum spanning tree algorithm revolves around checking if adding an edge creates a loop or not.

The most common way to find this out is an algorithm called Union Find. The Union Find algorithm divides the vertices into clusters and allows us to check if two vertices belong to the same cluster or not and hence decide whether adding an edge creates a cycle.

Algorithm 5.2: Kruskal Algorithm.

```
KRUSKAL(G):
A = ϕ
For each vertex v ∈ G.V:
   MAKE-SET(v)
For each edge (u, v) ∈ G.E ordered by increasing order by
weight(u, v):
   if FIND-SET(u) ≠ FIND-SET(v):
   A = A ∪ {(u, v)}
   UNION(u, v)
return A
```

Program 5.6: Implementation.

```
MAX = 30

class Edge:
   def __init__(self, u, v, w):
      self.u = u
      self.v = v
      self.w = w

class EdgeList:
   def __init__(self):
      self.data = []
      self.n = 0

elist = EdgeList()
spanlist = EdgeList()

# Graph adjacency matrix and vertex count
n = 6
Graph = [[0]*MAX for _ in range(MAX)]

def find(belongs, vertexno):
   return belongs[vertexno]

def applyUnion(belongs, c1, c2):
   for i in range(n):
      if belongs[i] == c2:
         belongs[i] = c1

def sort_edges():
   elist.data.sort(key=lambda edge: edge.w)

def kruskalAlgo():
   global elist, spanlist
   elist.data.clear()
   elist.n = 0
   # Build edge list from adjacency matrix (upper triangle only)
   for i in range(1, n):
```

```
        for j in range(i):
            if Graph[i][j] != 0:
                elist.data.append(Edge(i, j, Graph[i][j]))
                elist.n += 1

    sort_edges()

    belongs = [i for i in range(n)]
    spanlist.data.clear()
    spanlist.n = 0

    for edge in elist.data:
        cno1 = find(belongs, edge.u)
        cno2 = find(belongs, edge.v)

        if cno1 != cno2:
            spanlist.data.append(edge)
            spanlist.n += 1
            applyUnion(belongs, cno1, cno2)

def print_spanning_tree():
    cost = 0
    for edge in spanlist.data:
        print(f"{edge.u} - {edge.v} : {edge.w}")
        cost += edge.w
    print(f"Spanning tree cost: {cost}")

def main():
    # Define your graph matrix as per your C code
    # Only first 6x6 part used; rest of MAX ignored
    global Graph
    Graph = [[0]*MAX for _ in range(MAX)]

    Graph[0][0] = 0
    Graph[0][1] = 4
    Graph[0][2] = 4
    Graph[0][3] = 0
    Graph[0][4] = 0
    Graph[0][5] = 0

    Graph[1][0] = 4
    Graph[1][1] = 0
    Graph[1][2] = 2
    Graph[1][3] = 0
    Graph[1][4] = 0
    Graph[1][5] = 0

    Graph[2][0] = 4
    Graph[2][1] = 2
    Graph[2][2] = 0
    Graph[2][3] = 3
    Graph[2][4] = 4
    Graph[2][5] = 0

    Graph[3][0] = 0
    Graph[3][1] = 0
    Graph[3][2] = 3
    Graph[3][3] = 0
    Graph[3][4] = 3
    Graph[3][5] = 0

    Graph[4][0] = 0
    Graph[4][1] = 0
    Graph[4][2] = 4
    Graph[4][3] = 3
    Graph[4][4] = 0
    Graph[4][5] = 0

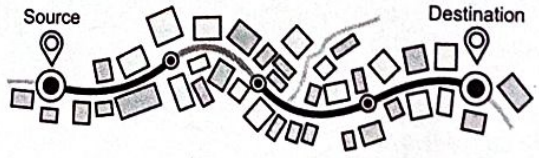
    Graph[5][0] = 0
    Graph[5][1] = 0
    Graph[5][2] = 2
    Graph[5][3] = 0
    Graph[5][4] = 3
    Graph[5][5] = 0

    kruskalAlgo()
    print_spanning_tree()

if __name__ == "__main__":
    main()
```

5.9 SHORTEST PATH [April 22, 24, Oct. 23, 24]

Dijkstra's Algorithm works on the basis that any subpath B → D of the shortest path A → D between vertices A and D is also the shortest path between vertices B and D.



■ the shortest path between the source and destination
■ a subpath which is also the shortest path between its source and destination

Fig. 5.13

Each Subpath is the Shortest Path

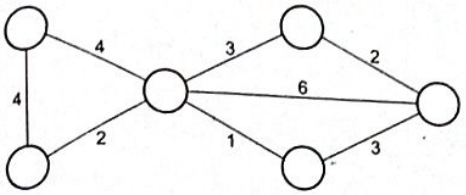
Djikstra use this property in the opposite direction i.e. we overestimate the distance of each vertex from the starting vertex. Then we visit each node and its neighbors to find the shortest subpath to those neighbors.

The algorithm uses a greedy approach in the sense that we find the next best solution hoping that the end result is the best solution for the whole problem.

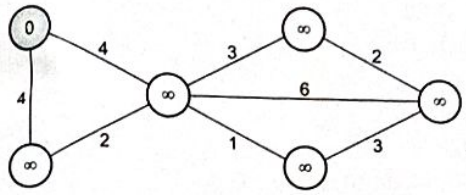
5.9.1 Example of Dijkstra's Algorithm

It is easier to start with an example and then think about the algorithm.

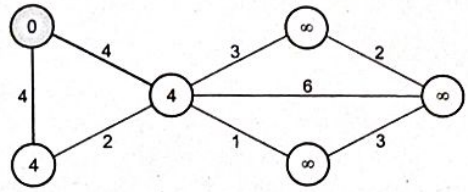
Step 1: Start with a weighted graph.



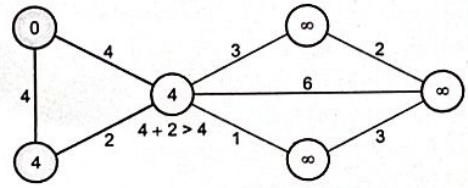
Step 2: Choose a starting vertex and assign infinity path values to all other devices.



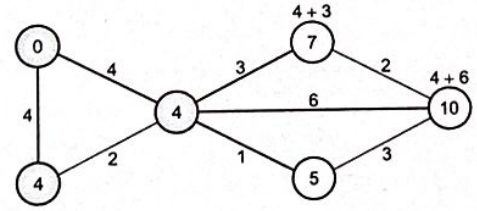
Step 3: Go to each vertex and update its path length.



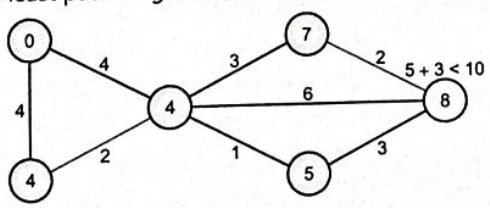
Step 4: If the path length of the adjacent vertex is lesser than new path length, don't update it.



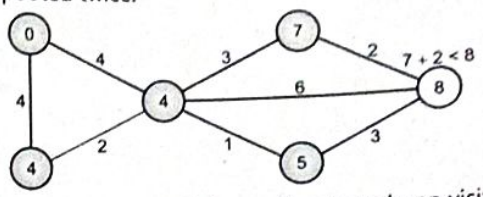
Step 5: Avoid updating path lengths of already visited vertices.



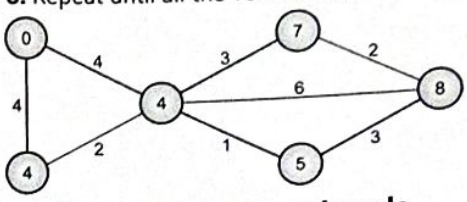
Step 6: After each iteration, we pick the unvisited vertex with the least path length. So, we choose 5 before 7



Step 7: Notice how the rightmost vertex has its path length updated twice.



Step 8: Repeat until all the vertices have been visited.



5.9.2 Djikstra's Algorithm Pseudocode

We need to maintain the path distance of every vertex. We can store that in an array of size v, where v is the number of vertices.

We also want to be able to get the shortest path, not only know the length of the shortest path. For this, we map each vertex to the vertex that last updated its path length.

Once the algorithm is over, we can back track from the destination vertex to the source vertex to find the path.

A minimum priority queue can be used to efficiently receive the vertex with least path distance.

Algorithm 5.3: Function dijkstra(G, S).

```
for each vertex V in G
    distance[V] <- infinite
    previous[V] <- NULL
    If V != S, add V to Priority Queue Q
distance[S] <- 0

while Q IS NOT EMPTY
    U <- Extract MIN from Q
    for each unvisited neighbour V of U
tempDistance<- distance[U] + edge_weight(U, V)
        if tempDistance< distance[V]
            distance[V] <- tempDistance
            previous[V] <- U
return distance[], previous[]
```

Program 5.7: Implementation.

```
INFINITY = 9999
MAX = 10

def dijkstra(Graph, n, start):
    cost = [[0]*n for _ in range(n)]
    distance = [0]*n
    pred = [0]*n
    visited = [False]*n
```

```
    # Create cost matrix
    for i in range(n):
        for j in range(n):
            if Graph[i][j] == 0:
                cost[i][j] = INFINITY
            else:
                cost[i][j] = Graph[i][j]
    for i in range(n):
        distance[i] = cost[start][i]
        pred[i] = start
        visited[i] = False

    distance[start] = 0
    visited[start] = True
    count = 1
    while count < n - 1:
        mindistance = INFINITY
        nextnode = -1
        for i in range(n):
            if distance[i] < mindistance and not visited[i]:
                mindistance = distance[i]
                nextnode = i

        if nextnode == -1:
            break  # No more reachable nodes

        visited[nextnode] = True

        for i in range(n):
            if not visited[i]:
                if mindistance + cost[nextnode][i] < distance[i]:
                    distance[i] = mindistance + cost[nextnode][i]
                    pred[i] = nextnode
        count += 1

    # Print distances
    for i in range(n):
        if i != start:
            print(f"Distance from source to {i}: {distance[i]}")

def main():
    n = 7
    Graph = [[0]*MAX for _ in range(MAX)]
    # Initialize the graph (only first n x n relevant)
    Graph[0][0] = 0
    Graph[0][1] = 0
    Graph[0][2] = 1
    Graph[0][3] = 2
    Graph[0][4] = 0
    Graph[0][5] = 0
    Graph[0][6] = 0

    Graph[1][0] = 0
    Graph[1][1] = 0
    Graph[1][2] = 2
    Graph[1][3] = 0
    Graph[1][4] = 0
    Graph[1][5] = 3
    Graph[1][6] = 0

    Graph[2][0] = 1
    Graph[2][1] = 2
    Graph[2][2] = 0
    Graph[2][3] = 1
    Graph[2][4] = 3
    Graph[2][5] = 0
    Graph[2][6] = 0

    Graph[3][0] = 2
    Graph[3][1] = 0
    Graph[3][2] = 1
    Graph[3][3] = 0
    Graph[3][4] = 0
    Graph[3][5] = 0
    Graph[3][6] = 1

    Graph[4][0] = 0
    Graph[4][1] = 0
    Graph[4][2] = 3
    Graph[4][3] = 0
    Graph[4][4] = 0
    Graph[4][5] = 2
    Graph[4][6] = 0

    Graph[5][0] = 0
    Graph[5][1] = 3
    Graph[5][2] = 0
    Graph[5][3] = 0
    Graph[5][4] = 2
    Graph[5][5] = 0
    Graph[5][6] = 1

    Graph[6][0] = 0
    Graph[6][1] = 0
    Graph[6][2] = 0
    Graph[6][3] = 1
    Graph[6][4] = 0
    Graph[6][5] = 1
    Graph[6][6] = 0

    start_node = 0
    dijkstra(Graph, n, start_node)

if __name__ == "__main__":
    main()
```

5.10 TREE

A tree is also one of the data structures that represent hierarchical data. Suppose we want to show the employees and their positions in the hierarchical form then it can be represented as shown below in Fig. 5.14:

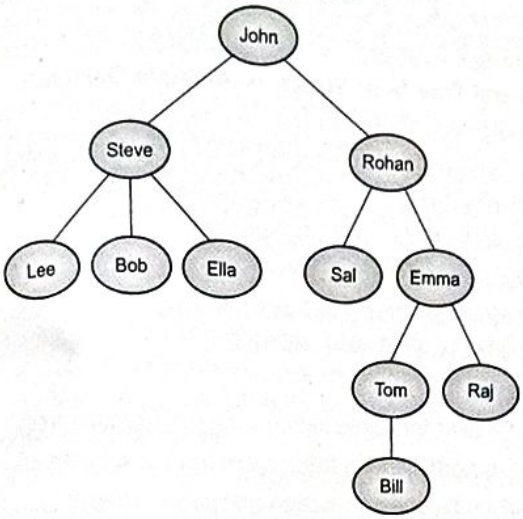


Fig. 5.14

The above tree shows the organization hierarchy of some company. In the above structure, john is the CEO of the company and John has two direct reports named as Steve and Rohan. Steve has three direct reports named Lee, Bob, Ella where Steve is a manager. Bob has two direct reports named Sal and Emma. Emma has two direct reports named Tom and Raj. Tom has one direct report named Bill. This particular logical structure is known as a Tree. Its structure is similar to the real tree, so it is named a Tree. In this structure, the root is at the top and its branches are moving in a downward direction. Therefore, we can say that the tree data structure is an efficient way of storing the data in a hierarchical way.

Let us understand some key points of the Tree Data Structure.

- A tree data structure is defined as a collection of objects or entities known as nodes that are linked together to represent or simulate hierarchy.
- A tree data structure is a non-linear data structure because it does not store in a sequential manner. It is a hierarchical structure as elements in a tree are arranged in multiple levels.
- In the tree data structure, the topmost node is known as a root node. Each node contains some data and data can be of any type. In the above tree structure, the node contains the name of the employee, so the type of data would be a string.
- Each node contains some data and the link or reference of other nodes that can be called children.

Some Basic Terms Used in Tree Data Structure

Let us consider the tree structure, which is shown in Fig. 5.15 below:

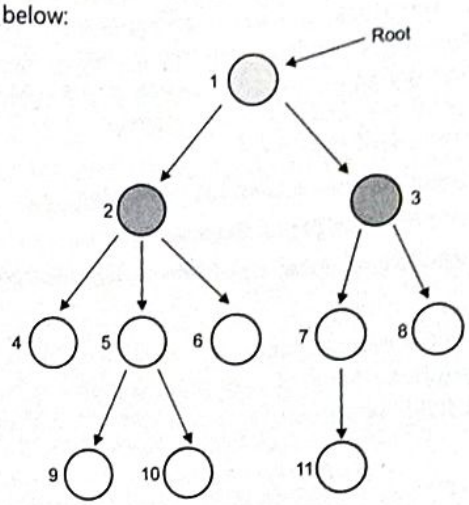


Fig. 5.15: Induction of trees

In the above structure, each node is labeled with some number. Each arrow shown in the above Fig. 5.15 is known as a link between the two nodes.

- **Root:** The root node is the topmost node in the tree hierarchy. In other words, the root node is the one that does not have any parent. In the above structure, node numbered 1 is the root node of the tree. If a node is directly linked to some other node, it would be called a parent-child relationship.
- **Child Node:** If the node is a descendant of any node, then the node is known as a child node.
- **Parent:** If the node contains any sub-node, then that node is said to be the parent of that sub-node.
- **Sibling:** The nodes that have the same parent are known as siblings.
- **Leaf Node:** The node of the tree, which does not have any child node, is called a leaf node. A leaf node is the bottom-most node of the tree. There can be any number of leaf nodes present in a general tree. Leaf nodes can also be called external nodes.
- **Internal Nodes:** A node has atleast one child node known as an internal.
- **Ancestor Node:** An ancestor of a node is any predecessor node on a path from the root to that node. The root node doesn't have any ancestors. In the tree shown in the above Fig. 5.15, nodes 1, 2 and 5 are the ancestors of node 10.
- **Descendant:** The immediate successor of the given node is known as a descendant of a node. In the above Fig. 5.15, 10 is the descendant of node 5.

5.11 TREE REPRESENTATION: SEQUENTIAL AND LINKED ORGANIZATION

In data structures, a tree is a non-linear hierarchical structure consisting of nodes connected by edges. The choice of tree representation in memory depends on the type of tree, expected operations, and memory constraints. Two common methods to represent trees are:

1. Sequential (Array-Based) Representation
2. Linked (Pointer-Based) Representation

1. Sequential Representation of Trees (Array-Based)

Definition:

In sequential representation, a tree is stored in an array where the position (index) of each node is used to determine the parent-child relationships based on a mathematical formula.

Use Case:

This method is best suited for complete binary trees, where all levels are fully filled except possibly the last.

Storage Rule (for Binary Tree):

Assume a node is stored at index i (starting from 1):

- Left child is at index 2i
- Right child is at index 2i + 1
- Parent is at index i / 2 (integer division)

Advantages

- Simple and efficient for complete or full binary trees.
- Fast access to parent and child nodes using index arithmetic.

Disadvantages

- Wastes memory for sparse trees or trees with missing children.
- Difficult to implement for general trees (non-binary).

Example:

For a binary tree:

```
    A
   / \
  B   C
 / \   \
D   E   F
```

Sequential (1-based index) representation:

Index: 1 2 3 4 5 6

Node: A B C D E F

2. Linked Representation of Trees (Pointer-Based)

Definition:

In the linked representation, each node contains the data and pointers (references) to its child nodes. This is a flexible and memory-efficient approach for representing general trees.

Types:

1. Binary Tree Node Structure:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```

2. General Tree Node Structure (Multiple Children):

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.first_child = None
        self.next_sibling = None
```

This structure (first child/next sibling) allows general trees with an arbitrary number of children.

Advantages

- Efficient for representing sparse or irregular trees.
- Supports general trees, n-ary trees, and binary trees.
- Dynamic memory usage, no space wastage.

Disadvantages

- Requires more complex memory management.
- Slight overhead due to pointers in each node.

Table 5.3 : Comparison Table

Feature	Sequential Representation	Linked Representation
Memory Usage	Fixed size; may waste space	Dynamic; memory allocated as needed
Suitable for	Complete Binary Trees	General trees, binary trees
Ease of Access	Fast (via index math)	Slower (via pointers)
Child Representation	By index calculation	By pointer fields
Flexibility	Limited (fixed structure)	Highly flexible

5.12 BINARY TREE

The Binary tree means that the node can have maximum two children. Here, binary name itself suggests that 'two' therefore, each node can have either 0, 1 or 2 children.

Let's understand the binary tree through an example.

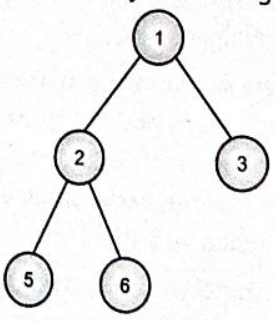


Fig. 5.16

The above tree Fig. 5.16 is a binary tree because each node contains the utmost two children. The logical representation of the above tree is given below in Fig. 5.17:

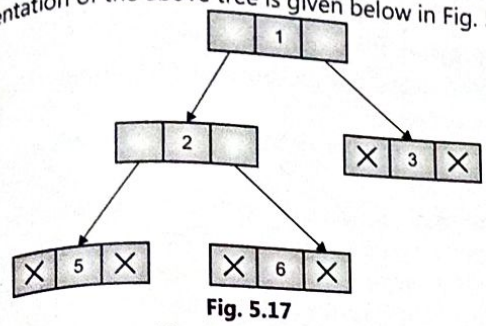


Fig. 5.17

In the above tree Fig. 5.16, node 1 contains two pointers, i.e., left and a right pointer pointing to the left and right node respectively. The node 2 contains both the nodes (left and right node); therefore, it has two pointers (left and right). The nodes 3, 5 and 6 are the leaf nodes, so all these nodes contain NULL pointer on both left and right parts.

Properties of Binary Tree:

At each level of i, the maximum number of nodes is 2i.

The height of the tree is defined as the longest path from the root node to the leaf node. The tree which is shown above Fig. 5.16 has a height equal to 3. Therefore, the maximum number of nodes at height 3 is equal to (1 + 2 + 4 + 8) = 15. In general, the maximum number of nodes possible at height h is (20 + 21 + 22 +....2h) = 2h+1 –1.

The minimum number of nodes possible at height h is equal to h+1. If the number of nodes is minimum, then the height of the tree would be maximum. Conversely, if the number of nodes is maximum, then the height of the tree would be minimum.

If there are 'n' number of nodes in the binary tree.

The Minimum Height Can be Computed as:

As we know that,

$$n = 2^{h+1} - 1$$

$$n+1 = 2^{h+1}$$

Taking log on both the sides,

$$\log_2(n+1) = \log 2(2^{h+1})$$

$$\log_2(n+1) = h+1$$

$$h = \log_2(n+1) - 1$$

The Maximum Height Can be Computed as:

As we know that,

$$n = h+1$$

$$h = n-1$$

Converting a General Tree to a Binary Tree

Introduction

A general tree is a tree where each node can have any number of children (not limited to two). However, many tree algorithms are defined only for binary trees, where each node has at most two children left and right.

To apply binary tree algorithms to general trees, we can convert a general tree into a binary tree using a method called the Left-Child Right-Sibling (LCRS) representation.

1. Left-Child Right-Sibling Representation (LCRS)

Definition:

The Left-Child Right-Sibling (LCRS) representation converts a general tree into a binary tree by:

- **Left Child:** Pointing to the node's first child.
- **Right Child:** Pointing to the node's next sibling.

This approach transforms a node with n children into a binary tree structure without losing the original hierarchy.

2. Conversion Rules

Let the general tree node be defined as:

```
MAX = 10  # or any desired maximum number of children

class GeneralNode:
  def __init__(self, data):
    self.data = data
    self.children = [None] * MAX
```

In LCRS binary tree, the equivalent node is:

```
class BinaryNode:
  def __init__(self, data):
    self.data = data
    self.left = None   # First child
    self.right = None  # Next sibling
```

5.13 STEPS TO CONVERT A GENERAL TREE TO A BINARY TREE [April 23]

Step 1: Start with the root node.

Step 2: The left pointer of a binary node points to its first child.

Step 3: The right pointer of a binary node points to its next sibling.

Step 4: Repeat recursively for all children.

Example: General Tree:

```
    A
   /|\
  B C D
   /\
  E F
```

LCRS Binary Tree Representation:

mathematica

CopyEdit

```
    A
   /
  B
   \
    C
     \
      D
     /
    E
     \
      F
```

Here:

A's left points to B
B's right points to C
C's right points to D
C's left points to E
E's right points to F

Advantages of LCRS Representation

- Enables application of binary tree algorithms (e.g., traversals) to general trees.
- Requires only two pointers per node, regardless of the number of children.
- Maintains parent-child and sibling relationships using binary tree structure.

Traversal in LCRS Binary Tree

- Preorder traversal of LCRS corresponds to the preorder of the general tree.
- Postorder traversal requires processing all children first (left-right chain), then the node.

Applications

- Parsing expressions (e.g., XML, HTML DOM trees).
- File system trees (directories and files).
- Compiler syntax trees.

5.14 RECURSIVE TRAVERSAL [April 23]

Tree traversal is the process of visiting each node in a tree, such as a binary tree or binary search tree, exactly once. There are several effective traversal algorithms which we will cover below. The tree we will be operating on looks like the following Fig. 5.18.

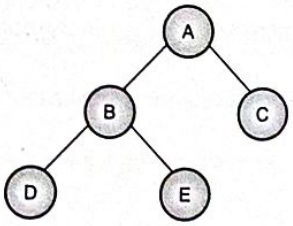


Fig. 5.18

And we can assume the tree is properly constructed via the following code which sets up nodes and links them to their proper child nodes:

```
class Node:
  def __init__(self, data):
    self.data = data
    self.left = None
    self.right = None
# Create nodes
root = Node('A')
n1 = Node('B')
n2 = Node('C')
n3 = Node('D')
n4 = Node('E')
# Setup children
root.left = n1
root.right = n2
n1.left = n3
n1.right = n4
```

A. Pre-order

A pre-order traversal on a tree performs the following steps starting from the root:

Step 1: Return the root node value.

Step 2: Traverse the left subtree by recursively calling the pre-order function.

Step 3: Traverse the right subtree by recursively calling the pre-order function.

For the tree above, performing a pre-order traversal would output the node values in the following order: A, B, D, E, C For the actual code implementation, we will be maintaining an array for the order of the nodes:

```
function pre_order(root, nodes) {
  nodes.push(root.data);
  if (root && root.left) {
    pre_order(root.left, nodes);
  }
  if (root && root.right) {
    pre_order(root.right, nodes);
  }
  return nodes;
}
```

Output:

pre_order(root, []); // => [A, B, D, E, C]

B. In-order

An in-order traversal on a tree performs the following steps starting from the root:

Step 1: Traverse the left subtree by recursively calling the in-order function.

Step 2: Return the root node value.

Step 3: Traverse the right subtree by recursively calling the in-order function.

For the tree above, performing an in-order traversal would output the node values in the following order: D, B, E, A, C

```
function post_order(root, nodes) {
  if (root && root.left) {
    post_order(root.left, nodes);
  }
  if (root && root.right) {
    post_order(root.right, nodes);
  }
  nodes.push(root.data);
  return nodes;
}
```

Output:

in_order(root, []); // => [D, B, E, A, C]

You can see that the only difference between the code for the in-order vs. pre-order traversal is where the appending of the node value is placed in the code. For post-order traversal below, that will be the only change as well. A good way to remember when to return the node value (or append the node value to an array) is, for pre-order do it first, for in-order

do it between the left and right traversal and as you'll see below, for post-order do it after traversing the left and right subtrees.

C. Post-Order

A post-order traversal on a tree performs the following steps starting from the root:

Step 1: Traverse the left subtree by recursively calling the post-order function.

Step 2: Traverse the right subtree by recursively calling the post-order function.

Step 3: Return the root node value.

For the tree above, performing a post-order traversal would output the node values in the following order: D, E, B, C, A

```
function post_order(root, nodes) {
  if (root && root.left) {
    post_order(root.left, nodes);
  }
  if (root && root.right) {
    post_order(root.right, nodes);
  }
  nodes.push(root.data);
  return nodes;
}
```

Output:

post_order(root, []); // => [D, E, B, C, A]

D. Level-Order

A level-order traversal on a tree performs the following steps starting from the root:

Step 1: Add the root to a queue.

Step 2: Pop the last node from the queue and return its value.

Step 3: Add all children of popped node to queue and continue from step 2 until queue is empty.

For the tree above, performing a level-order traversal would output the node values in the following order: A, B, C, D, E

```
function level_order(root, nodes) {
  var queue = [root];
  while (queue.length > 0) {
// front of queue is at element 0 and we push elements to back
of queue
    var n = queue.shift();
    nodes.push(n.data);
    if (n.left !== null) { queue.push(n.left); }
    if (n.right !== null) { queue.push(n.right); }
  }
  return nodes;
}
```

Output:

level_order(root, []); // => [A, B, C, D, E]

Applications of Tree Traversals

The algorithms above have several use cases in software and development. Below is a list of some of these common cases:

- To construct any binary tree, you need the in-order traversal array of nodes and either a pre-order or post-order array.
- A binary search tree can be constructed using only its pre-order traversal array.
- The in-order traversal of a binary search tree produces the elements in sorted order.
- You can perform a breadth-first search on a tree using a level-order traversal.

5.15 NON-RECURSIVE TRAVERSAL [April 23]

Algorithm 5.4: For Post-Order.

In this traversal first, traverse the leftmost subtree at the external node then visit the root node and lastly traverse the right subtree starting at the left external node.

POST-ORD(INFO, LEFT, RIGHT, ROOT)

Step-1 [Push NULL onto STACK and initialize PTR]
Set TOP=1, STACK[1]=NULL and PTR=ROOT.
Step-2 [Push left-most path onto STACK] repeat step 3 to 5 while PTR not equal NULL.
Step-3 set TOP=TOP+1 and STACK[TOP]=PTR. [Pushes PTR on STACK].
Step-4 if RIGHT[PTR] not equal NULL then [push on STACK.]
Set TOP=TOP+1 and STACK[TOP]= RIGHT[PTR]
[End of if structure]
Step-5 set PTR = LEFT[PTR].[Update pointer PTR]
[End of step 2 loop].
Step-6 set PTR= STACK[TOP] and TOP=TOP-1.
[Pops node from STACK].
Step-7 Repeat while PTR>0;
Apply PROCESS TO INFO[PTR].
Set PTR= STACK[TOP] and TOP= TOP-1.
[Pops node from STACK]
[End of loop]
Step-8 if PTR<0 then:
Set PTR= -PTR
Go to step 2
[End of if structure]
Step-9 Exit.

Algorithm 5.5: For In-Order.

In this traversal first traverse, the root node then traverses the left subtree of the external node and lastly traverse the right subtree of the external node.

IN-ORD(INFO, LEFT, RIGHT, ROOT)

Step-1 [Push NULL onto STACK and initialize PTR:]
Set TOP=1 STACK[1]= NULL and PTR=ROOT.
Step-2 Repeat while PTR not equal NULL[Pushes left most path onto STACK].
a. Set TOP=TOP+1 and STACK[TOP]=PTR [saves node]
b. Set PTR=LEFT[PTR] [updates PTR]
[End of loop]

Step-3 set PTR= STACK[TOP and TOP=TOP-1.[pops node from STACK].
Step-4 Repeat step 5 to 7 while PTR is not equal to Null; [backtracking].
Step-5 Apply PROCESS to INFO[PTR].
Step-6 [RIGHT child?] if RIGHT[PTR] is not equal NULL then.
a) Set PTR=RIGHT[PTR]
b) Go to step 3.[End of if structure.]
Step-7 set PTR= STACK[TOP] and TOP=TOP-1. [pops node]
[End of step 4 loop]
Step-8 Exit.

Algorithm 5.6: For Pre-Order.

In this traversal first, traverse all the left external node starting with the leftmost subtree which is then followed by bubble-up all the external node and then traverse the right subtree starting at the left external node which is the followed by bubble-up all the internal nodes and lastly visit the root node.

PRE-ORDER (INFO, LEFT, RIGHT, ROOT)

Step-1 [initially push NULL onto stack and initialize PTR.]
Set TOP=1 STACK[1]=NULL and PTR= ROOT.
Step-2 Repeat step 3 to 5 while PTR not equal NULL.
Step-3 Apply PROCESS to INFO[PTR].
Step-4 [RIGHT child?]
If RIGHT[PTR] not equal NULL then [PUSH on STACK]
Set TOP= TOP+1 and STACK[TOP]= RIGHT[PTR]
[End of if structure.]
Step-5 [LEFT child?]
If LEFT[PTR] not equal NULL then
Set PTR= LEFT[PTR].
Else [pop from STACK;]
Set PTR= STACK[TOP] and TOP=TOP+!;
[End of if structure]
[End of step 2 loop]
Step-6 Exit.

5.16 BINARY SEARCH TREE AND OPERATIONS [April 23 Oct. 23, 24]

Binary Search Tree ADT

Binary search tree is a binary tree in which every node X in the tree, the values of all the keys in its left sub tree are smaller than the key value in X and the values of all the keys in its right sub tree are larger than the key value in X.

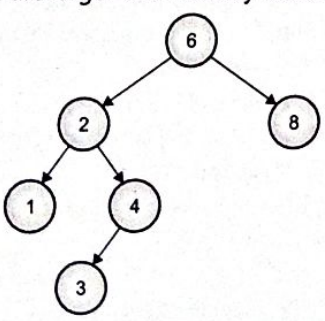


Fig. 5.19

Comparison Between Binary Tree and Binary Search Tree

Binary Tree	Binary Search Tree
A tree is said to be a binary tree if it has atmost two children. It does not have any order.	A binary search tree is a binary tree in which key values in the left node is less than the root and the root and the key values in the right node is greater than the root.

Declaration Routine for Binary Search Tree

```
struct TreeNode;
typedef struct Treenode *SearchTree; SearchTree Insert (int X, SearchTree T); SearchTree Delete (int X, SearchTree T); int Find (int X, SearchTree T);
int FindMin(SearchTree T); int FindMax(SearchTree T);
SearchTreeMakeEmpty(SearchTree T);
struct TreeNode
{
int Element; SearchTree Left; SearchTree Right;
};
```

MakeEmpty

This operation is mainly for initialization when the programmer prefer to initialize the first element as a one node tree.

Routine to Make an Empty Tree

```
def make_empty(tree):
  if tree is not None:
    tree.left = make_empty(tree.left)
    tree.right = make_empty(tree.right)
    tree = None
  return tree
```

Find

This operation requires returning a pointer to the node in tree T that has key X or NULL if there is no such node.
The find operation as follows:

- Check whether the root is NULL if so then return NULL.
- Otherwise, check the value X with the root node value (i.e. T->Element)
- If X is equal to T->Element, return T
- If X is less than T->Element, traverse the left of T recursively.
- If X is greater than T->Element, traverse the right of T recursively.

Routine for Find Operation

```
def find(x, tree):
  if tree is None:
    return None
  if x < tree.element:
    return find(x, tree.left)
  elif x > tree.element:
    return find(x, tree.right)
  else:
    return tree  # Element found
```

Example:

To find an element 4 (X = 4) in the below tree

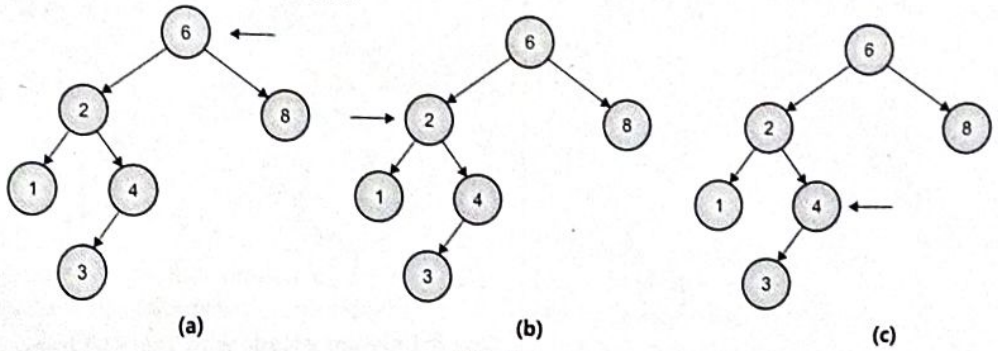


Fig. 5.20

In the first Fig. 5.20 (a), the element 4 is checked with root 6. 4 is less than 6. So goto the left subtree. In the second Fig. 5.20 (b), element 4 is checked with node 2. 4 is greater than 2. So goto the right subtree.

In the third Fig. 5.20 (c), the element 4 is checked with node 4. The element is equal. So return 4.

FindMin

This operation returns the position of the smallest element in the tree. To find the minimum element, start at the root and go left as long as there is a left child. The stopping point is the smallest element.

Recursive Routine for FindMin

```
def find_min(tree):
    if tree is None:
        return None
    elif tree.left is None:
        return tree
    else:
        return find_min(tree.left)
```

Non Recursive Routine for FindMin

```
def find_min(tree):
    if tree is not None:
        while tree.left is not None:
            tree = tree.left
    return tree
```

FindMax

This operation returns the position of the largest element in the tree. To find the maximum element, start at the root and go right as long as there is a right child. The stopping point is the largest element.

Recursive Routine for FindMax

```
def find_max(tree):
    if tree is None:
        return None
    elif tree.right is None:
        return tree
    else:
        return find_max(tree.right)
```

Non Recursive Routine for FindMax

```
def find_max(tree):
    if tree is not None:
        while tree.right is not None:
            tree = tree.right
    return tree
```

Insert

- To insert the element X into the tree, check with the root node T.
- If it is less than the root, traverse the left sub tree recursively until it reaches the T->Left equals to NULL. Then X is placed in T->Left.
- If it is greater than the root, traverse the right sub tree recursively until it reaches the T->Right equals to NULL. Then X is placed in T->Right.
- If X is found in the tree, do nothing.

Recursive Routine to Insert an Element into a Binary Search Tree

```
class TreeNode:
    def __init__(self, element):
        self.element = element
        self.left = None
        self.right = None
def insert(x, tree):
    if tree is None:
        return TreeNode(x)
    if x < tree.element:
        tree.left = insert(x, tree.left)
    elif x > tree.element:
        tree.right = insert(x, tree.right)
    # else: x is already in the tree, do nothing
    return tree
```

Example: To insert 8, 4, 1, 6, 5, 7, 10

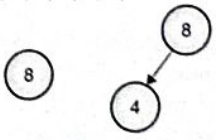
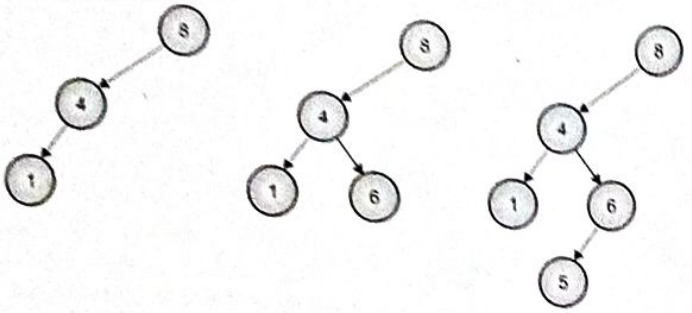


Fig. 5.21

First insert element 8 which is considered as root. Next insert element 4, 4 is less than 8, traverse towards left. Next insert element 1, 1<8 traverse towards left. 1<4 traverse towards left. To insert element 6, 6<8 traverse towards left.

6>4, place it as right child of 4. To insert element 5, 5<8 traverse towards left. 5>4, traverse towards right. 5<6, place it as left child of 6.



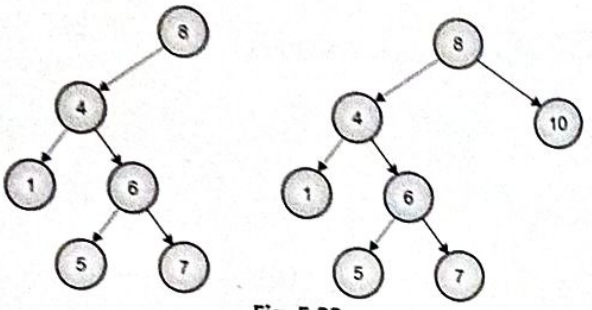


Fig. 5.22

To insert element 7, 7<8 traverse towards left. 7>4, traverse towards right. 7>6, place it as right child of 6.

To insert element 10, 10>8, place it as a right child of 8.

Delete

While deleting a node from a tree, the memory is to be released. Deletion operation is the complex operation in the binary search tree. To delete a node from the tree, consider the following three cases.

Node to be deleted is a leaf node

Node with one child.

Node with two children.

Case 1: Deleting the Leaf Node

Search the parent of the leaf node.

Make the parent link of the leaf node as NULL.

Release Memory.

Example:

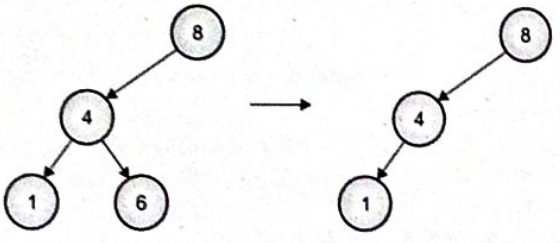


Fig. 5.23: Deletion of node 6

Case 2: Deleting the Node with Only One Child.

Search the parent of the node to be deleted.

Assign the parent link to the child node of the node to be deleted.

Release the memory of the deleted node.

If a node has one child, it can be deleted by adjusting its parent pointer to point to its child node.

Example: Deletion of a node with one child, before and after.

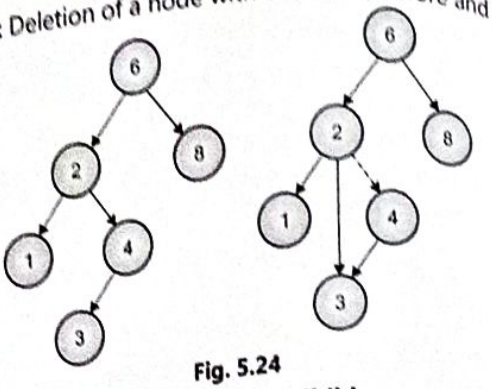


Fig. 5.24

Case 3: Deleting a Node with Two Children

The general strategy is to replace the data of the node to be deleted with its smallest data of the right sub tree (or) largest data of the left sub tree and recursively delete the data or node.

Inorder to delete a node with two children, it can be replaced by a node whose key is larger than any node in P's right sub tree to preserve the Binary Search Tree property. The possible nodes that could replace node P are

Rule 1: The node with the largest value in P"s left sub tree.

Rule 2: The node with the smallest value in P"s right sub tree.

Example: Deletion of a node 4 with two children, before and after.

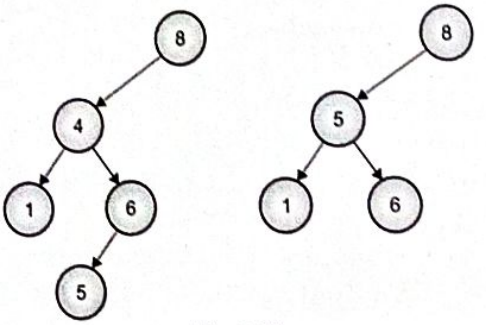


Fig. 5.25

Deletion Routine for Binary Search Tree

```
class TreeNode:
  def __init__(self, element):
    self.element = element
    self.left = None
    self.right = None
def find_min(tree):
  while tree.left is not None:
    tree = tree.left
  return tree
def delete(x, tree):
  if tree is None:
    print("Element not found")
    return None
  if x < tree.element:
    tree.left = delete(x, tree.left)
  elif x > tree.element:
    tree.right = delete(x, tree.right)
  else:  # Found node to delete
    if tree.left and tree.right:  # Two children
      tmp = find_min(tree.right)
```

```
        tree.element = tmp.element
        tree.right = delete(tmp.element, tree.right)
    else: # One or zero child
        if tree.left is None:
            return tree.right
        elif tree.right is None:
            return tree.left
    return tree
```

CASE STUDY

1. GPS/Navigation system that models a city map as a weighted graph and applies core graph algorithms

Problem Statement

Design a system to represent a city map where intersections are nodes and roads are weighted edges (with distance, traffic delay, etc.). Implement shortest path algorithms to compute the most efficient route from a source to a destination.

Real-World Context

Applications like Google Maps or Waze compute shortest or fastest paths using real-time road network graphs. Each node is a location, and each edge is a road segment with weights such as:

- Distance
- Estimated time (ETT)
- Traffic weight

Data Structures Used

- **Weighted Directed Graph** (Adjacency List)
- **Priority Queue / Min-Heap** for Dijkstra's algorithm

```
import heapq
import sys
class CityMap:
    def __init__(self, n):
        self.num_locations = n
        self.adj_list = [[] for _ in range(n)]
    def add_road(self, from_node, to_node, weight):
        self.adj_list[from_node].append((weight, to_node))
    def dijkstra(self, source):
        dist = [sys.maxsize] * self.num_locations
        dist[source] = 0
        pq = [(0, source)]  # (distance, node)
        while pq:
            d, u = heapq.heappop(pq)

            for weight, v in self.adj_list[u]:
                if dist[v] > d + weight:
                    dist[v] = d + weight
                    heapq.heappush(pq, (dist[v], v))
        print(f"Shortest distances from node {source}:")
        for i in range(self.num_locations):
            print(f"To {i} = {dist[i]} units")
# --- Main ---
if __name__ == "__main__":
    city_map = CityMap(6)
    city_map.add_road(0, 1, 5)
    city_map.add_road(0, 2, 2)
    city_map.add_road(1, 2, 1)
    city_map.add_road(1, 3, 7)
    city_map.add_road(2, 3, 4)
    city_map.add_road(3, 4, 1)
    city_map.add_road(4, 5, 3)

    city_map.dijkstra(0)
```

Graph Algorithm Used

Dijkstra's Algorithm

- Greedy method
- Best for single-source shortest paths with non-negative weights

Time Complexity

- Dijkstra (using Min-Heap): O(E log V)

Enhancements

- Add A* heuristic for real-time navigation
- Integrate dynamic edge weights using live traffic
- Visualize route using UI components

Takeaways

Graphs provide a natural structure to model cities. Algorithms like Dijkstra's ensure scalable and accurate route computations for real-time navigation.

2. ZIP/GZIP File Compression using Huffman Coding (Frequency-Based Encoding)

Title

Lossless Data Compression using Huffman Tree Algorithm

Problem Statement

How can we compress a file or string to reduce size without losing any information? Use Huffman encoding, which assigns shorter binary codes to more frequent characters.

Real-World Context

Tools like ZIP, GZIP, and PNG internally use Huffman coding to reduce file size efficiently. Huffman Trees are a classic application of binary trees and priority queues in compression.

Data Structures Used

- **Min-Heap** (Priority Queue)
- **Binary Tree** (Huffman Tree)
- **Map** for character-to-code mapping

```
import heapq
from collections import defaultdict
class Node:
    def __init__(self, ch, freq):
        self.ch = ch
        self.freq = freq
        self.left = None
        self.right = None
    # For priority queue comparison
    def __lt__(self, other):
        return self.freq < other.freq
def generate_codes(root, code, codes):
    if root is None:
```

```
        return
    if root.left is None and root.right is None:
        codes[root.ch] = code
    generate_codes(root.left, code + "0", codes)
    generate_codes(root.right, code + "1", codes)
def huffman_encoding(text):
    # Step 1: Frequency dictionary
    freq = defaultdict(int)
    for ch in text:
        freq[ch] += 1
    # Step 2: Priority queue
    heap = []
    for ch, f in freq.items():
        heapq.heappush(heap, Node(ch, f))
    # Step 3: Build Huffman Tree
    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        merged = Node('\0', left.freq + right.freq)
        merged.left = left
        merged.right = right
        heapq.heappush(heap, merged)
    # Step 4: Generate codes
    root = heapq.heappop(heap)
    codes = {}
    generate_codes(root, "", codes)
    # Step 5: Display codes
    print("\nHuffman Codes:")
    for ch, code in codes.items():
        print(f"{repr(ch)}: {code}")
    # Step 6: Print encoded output
    print("\nEncoded Output:")
    encoded = ''.join(codes[ch] for ch in text)
    print(encoded)
# --- Main ---
if __name__ == "__main__":
    data = "huffman compression example"
    huffman_encoding(data)
```

Step-by-Step Process

Step 1: Count frequency of each character.

Step 2: Build a priority queue (min-heap).

Step 3: Construct a Huffman Tree using these frequencies.

Step 4: Traverse the tree to assign binary codes.

Step 5: Encode the input string using those codes.

Complexity Analysis

- **Time Complexity:** O(n log n), where n is number of characters
- **Space Complexity:** O(n) for tree + O(n) for codes

Enhancements

- Store Huffman Tree in compressed file header for decoding.
- Implement decoding function for full decompression.

Takeaways

Huffman encoding is a perfect use case for combining trees and heaps to solve a real-world problem efficiently. It teaches:

- Priority-based selection
- Tree traversal
- Greedy optimization

EXERCISE

1. Define a graph and explain the basic terms associated with it (vertices, edges, degree, path, cycle, connected graph). Also describe how graphs are represented in memory.
2. What is the difference between an adjacency matrix and an adjacency list? Write the advantages and disadvantages of each.
3. Describe Depth-First Search (DFS) and Breadth-First Search (BFS) traversal techniques for a graph. Provide pseudocode and an example for each.
4. What is a Minimum Spanning Tree (MST)? Why is it useful in real-world applications like networking?
5. Explain Prim's Algorithm for finding the Minimum Spanning Tree using a suitable example and step-by-step explanation.
6. Explain Kruskal's Algorithm for computing MST. Compare it with Prim's algorithm in terms of approach and data structures used.
7. What is a general tree? Describe its sequential and linked representation. How can a general tree be converted to a binary tree?
8. Define a binary tree. What are its properties? Write recursive and non-recursive algorithms for inorder, preorder, and postorder traversals.
9. What is a Binary Search Tree (BST)? List and explain the basic operations that can be performed on a BST such as insertion, deletion, and search.
10. Case Study: Explain how a GPS/Navigation system can be modeled using a weighted graph. Also, describe how Huffman coding is used in ZIP/GZIP file compression using frequency-based encoding.

✠ ✠ ✠